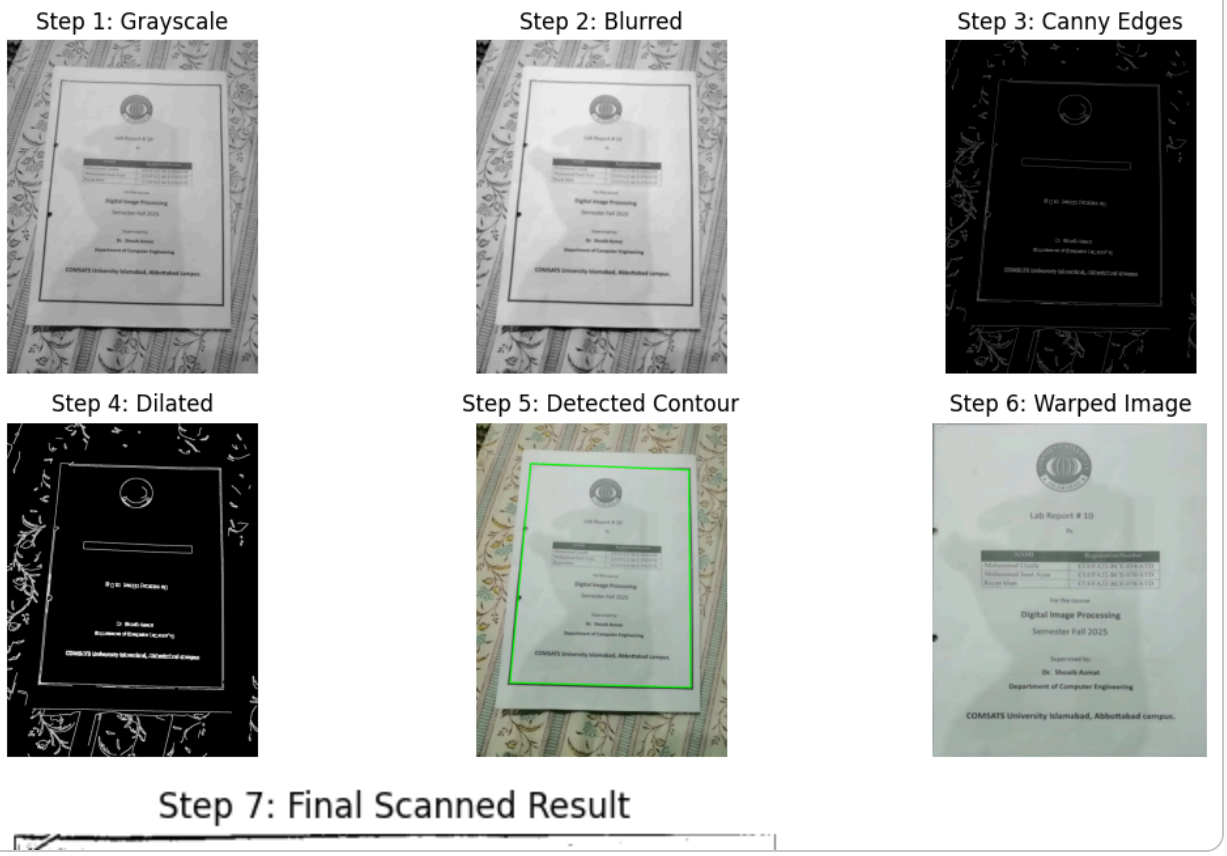
```
!pip install streamlit opencv-python-headless numpy Pillow pyngrok
!npm install -g localtunnel
```

Show hidden output

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# --- Helper Functions ---
def order_points(pts):
    rect = np.zeros((4, 2), dtype="float32")
    s = pts.sum(axis=1)
    rect[0] = pts[np.argmin(s)]
    rect[2] = pts[np.argmax(s)]
    diff = np.diff(pts, axis=1)
    rect[1] = pts[np.argmin(diff)]
    rect[3] = pts[np.argmax(diff)]
    return rect

def four_point_transform(image, pts):
    rect = order_points(pts)
    (tl, tr, br, bl) = rect
    widthA = np.sqrt(((br[0] - bl[0]) ** 2) + ((br[1] - bl[1]) ** 2))
    widthB = np.sqrt(((tr[0] - tl[0]) ** 2) + ((tr[1] - tl[1]) ** 2))
    maxWidth = max(int(widthA), int(widthB))
    heightA = np.sqrt(((tr[0] - br[0]) ** 2) + ((tr[1] - br[1]) ** 2))
    heightB = np.sqrt(((tl[0] - bl[0]) ** 2) + ((tl[1] - bl[1]) ** 2))
    maxHeight = max(int(heightA), int(heightB))
    dst = np.array([[0, 0],
                    [maxWidth - 1, 0],
                    [maxWidth - 1, maxHeight - 1],
                    [0, maxHeight - 1]], dtype="float32")
    M = cv2.getPerspectiveTransform(rect, dst)
    return cv2.warpPerspective(image, M, (maxWidth, maxHeight))

# --- Execution ---
image_path = "/content/img_04.jpg"
image = cv2.imread(image_path)

if image is None:
    print("Error: Image not found.")
else:
    orig = image.copy()

    # Step 1: Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    plt.figure(figsize=(12, 6))
    plt.subplot(2, 3, 1)
    plt.imshow(gray, cmap='gray')
    plt.title("Step 1: Grayscale")
    plt.axis('off')

    # Step 2: Blur to reduce noise
    blurred = cv2.bilateralFilter(gray, 9, 75, 75)
    plt.subplot(2, 3, 2)
    plt.imshow(blurred, cmap='gray')
```

```
        plt.title("Step 2: Blurred")
        plt.axis('off')

        # Step 3: Detect edges
        edged = cv2.Canny(blurred, 30, 150)
        plt.subplot(2, 3, 3)
        plt.imshow(edged, cmap='gray')
        plt.title("Step 3: Canny Edges")
        plt.axis('off')

        # Step 4: Dilate edges to close gaps
        kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
        dilated = cv2.dilate(edged, kernel, iterations=1)
        plt.subplot(2, 3, 4)
        plt.imshow(dilated, cmap='gray')
        plt.title("Step 4: Dilated")
        plt.axis('off')

        # Step 5: Find contours and draw the biggest 4-point contour
        cnts, _ = cv2.findContours(dilated.copy(), cv2.RETR_LIST, cv2.CHAIN_APPROX_
        cnts = sorted(cnts, key=cv2.contourArea, reverse=True)[:10]

        screenCnt = None
        for c in cnts:
            peri = cv2.arcLength(c, True)
            approx = cv2.approxPolyDP(c, 0.02 * peri, True)
            if len(approx) == 4:
                screenCnt = approx
                break

        if screenCnt is None:
            print("Still couldn't detect corners. Check your background contrast."
        else:
            image_with_contour = orig.copy()
            cv2.drawContours(image_with_contour, [screenCnt], -1, (0, 255, 0), 5)
            plt.subplot(2, 3, 5)
            plt.imshow(cv2.cvtColor(image_with_contour, cv2.COLOR_BGR2RGB))
            plt.title("Step 5: Detected Contour")
            plt.axis('off')

            # Step 6: Warp the image (Perspective Transform)
            warped = four_point_transform(orig, screenCnt.reshape(4, 2))
            plt.subplot(2, 3, 6)
            plt.imshow(cv2.cvtColor(warped, cv2.COLOR_BGR2RGB))
            plt.title("Step 6: Warped Image")
            plt.axis('off')

            plt.tight_layout()
            plt.show()

            # Step 7: Final scanned effect (Black & White)
            warped_gray = cv2.cvtColor(warped, cv2.COLOR_BGR2GRAY)
            scanned = cv2.adaptiveThreshold(warped_gray, 255,
                                            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                            cv2.THRESH_BINARY, 11, 2)

            plt.figure(figsize=(6, 6))
            plt.imshow(scanned, cmap='gray')
            plt.title("Step 7: Final Scanned Result")
```

```
plt.axis('off')
plt.show()
```



Step 7: Final Scanned Result